Meshtastic LoRa Room Server

Final Project Report

INFO4 - Polytech Grenoble

Academic year 2025-2026

Authors	TAWFIK Adam UNAY Arhan
Supervisor	Francois LETELLIER
Project start	January 2026
Date	March 2026

This report summarizes the design, implementation, validation, and project management of a Room Server built on top of the Meshtastic LoRa mesh network.

Abstract

This project focuses on the design and implementation of a Room Server for the Meshtastic LoRa mesh network. The system runs on a Raspberry Pi 3B connected to a Seeed Studio Wio-E5 Mini radio module and provides an application layer able to create rooms, store messages, and return concise replies adapted to LoRa constraints. Users interact with the server through text commands such as /room create, /room post, and /room read. The project addresses several technical challenges: persistent storage, low-bandwidth communication, limited packet payload, potential packet loss, and robustness after unexpected power loss. To answer these constraints, we selected a modular Python architecture, an SQLite database for persistent local storage, a private Meshtastic channel for isolated tests, and a systemd service to automatically restart the server on boot or after failure. The report presents the project context, requirements, work breakdown structure, technical choices, implementation details, risk management, validation strategy, and lessons learned during development.

Keywords: LoRa, Meshtastic, Raspberry Pi, SQLite, embedded systems, low-bandwidth networking, project management.

Table of Contents

Abstract....2

Table of Contents....3

1. Introduction....5

2. Project Context, Needs, and Objectives....5

2.1 Context and identified need....5

2.2 Main objectives and expected deliverables....6

2.3 Functional perimeter....6

3. Project Management Approach....7

3.1 Development method....7

3.2 Roles and collaborative tools....7

3.3 Stakeholder analysis....7

3.4 SWOT summary....8

4. Work Breakdown Structure and Planning....9

4.1 Work breakdown structure (WBS)....9

4.2 Planned and actual timeline....9

4.3 Time and cost overview....10

5. Quality Criteria, Risks, and Decision Trade-offs....11

5.1 Quality criteria....11

5.2 Cost-delay-quality triangle....11

5.3 Risk analysis and mitigation....11

6. Technical Architecture and Design Choices....13

6.1 Hardware and software stack....13

6.2 Logical architecture....14

6.3 Main modules....15

6.4 Task Allocation and Main Contributions....17

6.5 Lines of Code and Implementation Volume....17

6.6 Persistence strategy....18

7. Implementation Details..... 19
7.1 Command model..... 19
7.2 Sequence of a message posting operation..... 19
7.3 LoRa-specific optimizations..... 19
7.4 Deployment and operational robustness..... 20
8. Tests, Validation, and Observed Results..... 21
8.1 Validation strategy..... 21
8.2 Main test scenarios..... 21
8.3 Limits of the validation campaign..... 21
8.4 What was effectively demonstrated..... 22
9. Impacts, Lessons Learned, and Future Work..... 23
9.1 Project impacts..... 23
9.2 Lessons learned..... 23
9.3 Possible extensions..... 23
10. Conclusion..... 24
Bibliography and Sources..... 25
Appendices..... 26
Appendix A - Example command set..... 26
Appendix B - Main deliverables..... 26

1. Introduction

LoRa networks are designed for long-range communication with low power consumption and limited data throughput. They are particularly relevant for Internet of Things applications deployed in environments where conventional network infrastructure is unavailable, unreliable, or too costly. Meshtastic builds a decentralized mesh layer on top of LoRa and enables text-based communication between nodes without relying on the Internet.

However, the default Meshtastic workflow mainly relies on broadcast messaging. In practice, this means that messages are shared across the network without native support for persistent chat rooms, structured message history, or advanced conversation organization. The engineering challenge of this project was therefore to create a specialized node, called a Room Server, able to add these missing features while preserving compatibility with the existing Meshtastic ecosystem.

The report is organized in two complementary dimensions. The first one documents the technical product: hardware setup, software architecture, command processing, persistence strategy, deployment, and validation. The second one documents how the project was managed: work breakdown structure, planning, team organization, quality criteria, risks, and lessons learned.

2. Project Context, Needs, and Objectives

2.1 Context and identified need

The subject provided by the supervisor required the design of a Meshtastic Room Server able to receive commands through LoRa messages, manage chat rooms, and keep message history persistent after reboot. The server had to be built with portable software, off-the-shelf hardware, and a robust storage strategy suitable for an embedded environment. The Room Server was expected to operate above Meshtastic, without modifying the Meshtastic protocol itself.

The need can be summarized as follows: users should be able to organize exchanges into logical rooms, retrieve message history on demand, and rely on the service even after abrupt shutdown. This need is particularly meaningful in low-infrastructure or emergency communication contexts, where a compact and autonomous messaging service may bring practical value.

2.2 Main objectives and expected deliverables

- Provide a command-based interface: room creation, room listing, message posting, and room reading.
- Store rooms and messages persistently on the server side.
- Deploy the application on a Raspberry Pi connected to a Meshtastic-compatible LoRa radio.
- Demonstrate restart robustness after server reboot or USB disconnection.
- Produce technical documentation, project management documentation, and a final report.

2.3 Functional perimeter

Function	Description	Status in project scope
Room creation	Create a room identified by a unique name.	Implemented
Room listing	Return the existing rooms stored on the server.	Implemented
Message publication	Store a new message inside a selected room.	Implemented
History reading	Read the latest N messages from a room.	Implemented with limits
Help command	Provide a concise command summary.	Implemented
Persistence after reboot	Keep rooms and messages after restart.	Implemented
User rights / access control	Advanced permissions and restricted rooms.	Not implemented

The table above distinguishes the validated core features from optional extensions. Advanced administration, user rights, or cross-server synchronization were considered outside the minimum perimeter and are discussed later as possible extensions.

3. Project Management Approach

3.1 Development method

The project followed a lightweight iterative approach close to agile development. Instead of building the whole system in one pass, we advanced through short weekly iterations driven by technical milestones: hardware acquisition, database initialization, command parsing, LoRa integration, robustness improvements, and final documentation. Each week ended with a progress report describing completed tasks, issues, and next objectives.

3.2 Roles and collaborative tools

Area	Main contribution	Tools used
Backend and architecture	Command parser, business logic, data model, architecture documentation.	Python, Markdown, Git
Hardware and system integration	Raspberry Pi setup, USB/serial communication, systemd service, field tests.	Linux, Meshtastic CLI, journalctl
Documentation and reporting	Weekly reports, meeting report, final report, planning and slides.	LibreOffice / Word, Git, PDF

The repository, technical notes, weekly progress reports, and meeting report formed the shared project memory. During the temporary GitLab outage, a private repository was used as an intermediate solution before migration to the school environment.

3.3 Stakeholder analysis

Stakeholder	Role	Expectation	Impact on project
Students	Design and implement the solution	Deliver a working demonstrator and documentation	Direct execution
Supervisor	Validate direction and evaluate results	Technical coherence and project management	High

		discipline	
FabLab / hardware providers	Provide Raspberry Pi and LoRa modules	Availability of working hardware	Medium
Future users / evaluators	Test the demonstrator	Simple commands, reliable replies, persistent history	Medium

3.4 SWOT summary

Strengths Modular Python architecture; simple command model; persistent storage with SQLite; realistic embedded deployment.	**Weaknesses** Limited radio bandwidth; dependency on physical hardware; restricted time for extensive performance benchmarking.
Opportunities Use in emergency or infrastructure-limited communication scenarios; future web interface or rights management.	**Threats** Packet loss, USB disconnection, power instability, and environmental variability during radio tests.

4. Work Breakdown Structure and Planning

4.1 Work breakdown structure (WBS)

WBS ID	Task	Main output	Notes
1	Project framing and requirement analysis	Problem statement, milestones	Based on subject sheet and kickoff discussions
2	Hardware preparation	Raspberry Pi and Wio-E5 available	Included power supply troubleshooting
3	Persistence layer	SQLite schema and storage class	rooms and messages tables
4	Command processing	parser.py and room_manager.py	Business logic for /room commands
5	Communication layer	meshtastic_comm_hw.py and client.py	Real hardware communication and client simulator
6	Robustness layer	logger.py, reset_db.py, systemd service	Restart, monitoring, recovery
7	Validation and documentation	Weekly reports, tests, final report, flyer	Project management traceability

4.2 Planned and actual timeline

The project was organized as an incremental sequence of milestones. The initial planning was refined each week through progress reports. The figure below summarizes the timeline used to monitor the project.

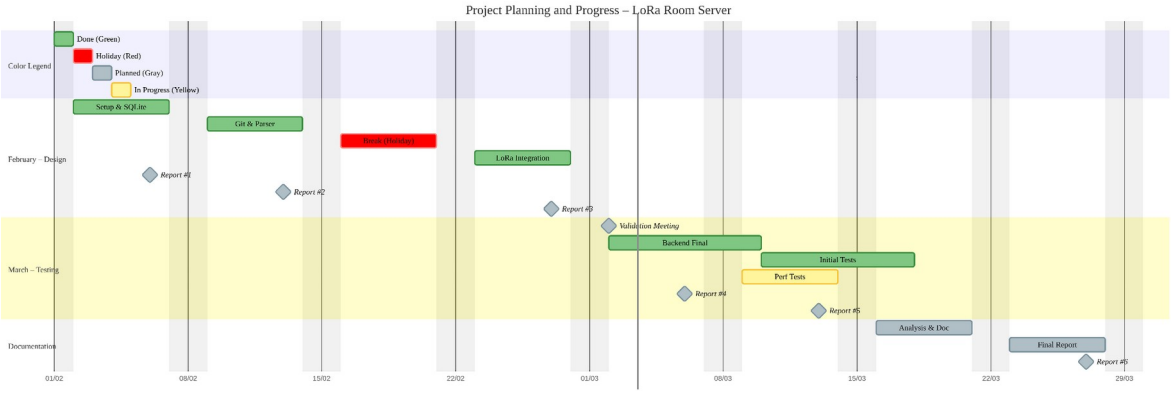

Figure 1 - Project planning and timeline used for weekly monitoring.

The timeline highlights a progression from problem analysis and database initialization toward hardware integration, robustness work, and final documentation. The main gap between forecast and reality came from hardware-related issues, especially the Raspberry Pi power supply and the need to stabilize radio communication before large-scale testing.

4.3 Time and cost overview

Resource	Status	Comment
Hardware budget	Provided by laboratory	Raspberry Pi 3B and Wio-E5 modules were supplied for the project.
Software budget	No additional license cost	Python, SQLite, Meshtastic CLI, and Linux tools were open or already available.
Human workload	Tracked through weekly progression	Work volume was mainly measured through milestones, implementation tasks, and reporting.

A precise hour-by-hour timesheet was not maintained, but the project did include continuous weekly reporting. This means that the human cost can be described qualitatively through the documented iterations, the number of implemented modules, and the validation activities.

5. Quality Criteria, Risks, and Decision Trade-offs

5.1 Quality criteria

Criterion	Reason	How it was addressed
Modularity	Facilitates debugging and maintenance	Separate modules for parsing, room management, communication, database, and logging.
Persistence integrity	Server state must survive restart	SQLite chosen instead of a flat file because it supports transactions.
Operational robustness	Embedded hardware can fail or disconnect	systemd auto-restart and hardware path monitoring.
Readable user interaction	LoRa replies must remain short and clear	Concise command syntax, direct acknowledgements, and message chunking.
Traceability	Project decisions must be explainable	Weekly reports, meeting report, and technical architecture note.

5.2 Cost-delay-quality triangle

The project constantly balanced three priorities. The first one was quality, because persistence and reboot robustness were mandatory. The second one was schedule, as the project had to reach a demonstrable state within the semester. The third one was cost, which remained limited because the main hardware was provided by the laboratory. This context justified several decisions: focusing on a reliable local SQLite storage rather than a more complex distributed database, using simple text commands instead of a richer but heavier interface, and prioritizing real hardware validation over optional feature expansion.

5.3 Risk analysis and mitigation

Risk	Potential impact	Mitigation	Observed status

Insufficient Raspberry Pi power supply	Server unavailable	Replace the adapter and validate boot stability	Observed and solved
LoRa payload size limitations	Replies truncated or lost	Reduce payload size and split longer content	Observed and mitigated
USB radio disconnection	Process hang or no communication	Monitor hardware presence and use systemd restart	Observed and mitigated
Public channel interference	Noise from external users	Configure a private channel with a dedicated PSK	Observed and mitigated
Limited time for benchmarking	Incomplete quantitative performance analysis	Prioritize demonstrator validity and core tests	Partially accepted

6. Technical Architecture and Design Choices

6.1 Hardware and software stack

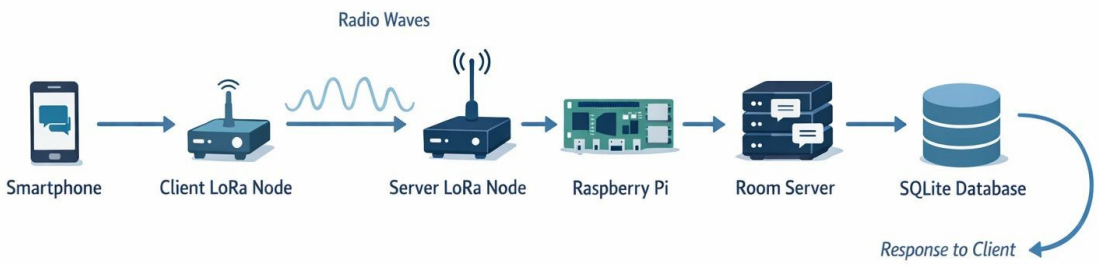

Figure 2 - End-to-end system architecture from the smartphone client to the Room Server and SQLite database.

This architecture diagram summarizes the complete data path of the demonstrator. A user sends a command from the smartphone through a client LoRa node. The packet crosses the LoRa link toward the server node connected to the Raspberry Pi, where the Room Server processes the request and stores or retrieves data from the SQLite database before sending a response back to the client.

Layer	Selection
Server hardware	Raspberry Pi 3B
Radio module	Seeed Studio Wio-E5 Mini
Operating system	Raspberry Pi OS / Linux
Main language	Python 3
Persistence	SQLite

Communication framework	Meshtastic Python tooling and serial interface

6.2 Logical architecture

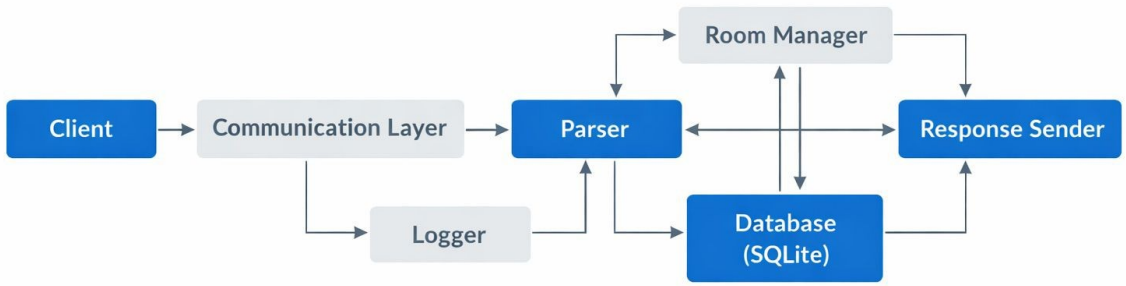

Figure 3 - Software architecture showing the interaction between the communication layer, parser, Room Manager, response sender, logger, and SQLite database.

The software architecture deliberately separates transport concerns from business logic and persistence. The communication module receives packets from the Wio-E5 radio, the parser identifies valid commands, the RoomManager applies the corresponding business rules, and the database module stores or retrieves persistent data. Additional operational modules support logging, reset, and restart supervision.

The class diagram complements the previous logical view by detailing the main software classes used in the implementation. It highlights the central role of RoomManager, which coordinates command handling, storage access, and response production. The diagram also shows the interaction between TransportHardware, IncomingMessage, OutgoingMessage, CommandParser, StorageSQLite, and the global logger. This view is useful to understand how responsibilities are distributed in the code and how modularity was preserved throughout the implementation.

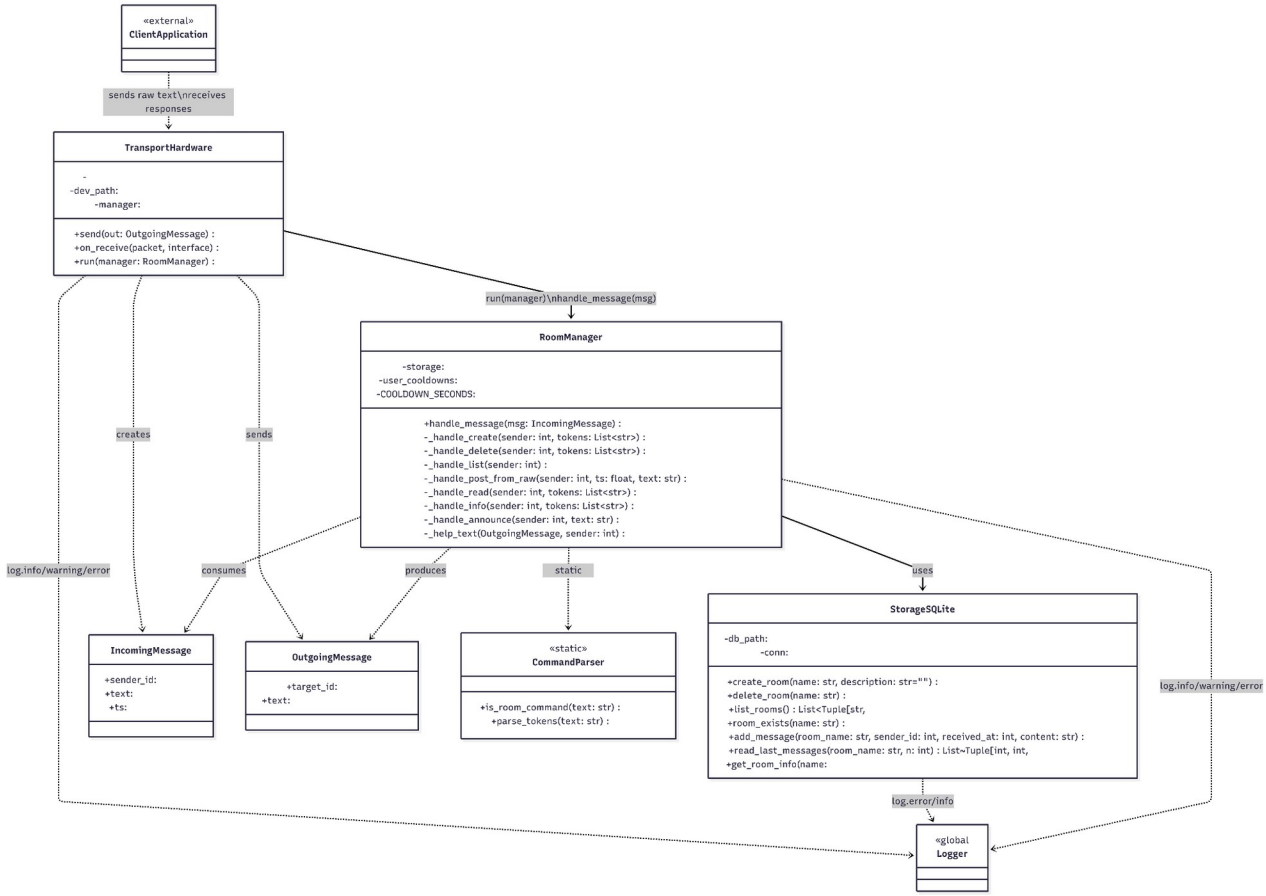

Figure 4 - UML class diagram of the main software entities and their relationships.

6.3 Main modules

File/Module	Main Role	Technical Descroption
main.py	**Entry Point (Orchestrator)**	This is the file executed to start the server. It initializes the database, the`RoomManager`, and connects the hardware.
parser.py	**Syntax Parser (Translator)**	Parses `/room` commands, extracts tokens, and helps the server distinguish actions and arguments before execution.
room_manager.py	**Business Logic (Brain)**	**Central module** of the server. It processes

		incoming commands, manages rooms, applies anti-spam protection, formats long replies, and coordinates all interactions with the SQLite storage layer.
database.py	**Data-Management (Storage)**	Manages the **SQLite** connection. It creates the tables (`rooms`, `messages`) and contains functions to insert or read data (CRUD).
meshtastic_comm_hw.py	**Hardware Interface (Real Driver)**	This is the Wio-E5 "driver". It listens to the USB port (`/dev/ttyUSB0`), captures incoming LoRa signals, and sends responses.
`meshtastic_comm.py`	**Simulation Interface (Mock)**	Simulates the LoRa interface via the console. Allows testing the entire server logic without connected hardware, using the keyboard as input and screen as output.
client.py	**Client Simulator (Tester)** computer	The script used on the test computer. It allows sending messages to the server via a second LoRa module, simulating a real user.
logger.py	**Logging (Tracker)**	Provides logging support across the application for debugging, monitoring,

		warnings, and error tracking.
reset_db.py	**Maintenance Tool (Cleaner)**	A small utility script to cleanly delete the `.db` file and reset the database to zero in case of problems.

6.4 Task Allocation and Main Contributions

The implementation work was distributed between the two team members according to the project needs and the progression of the development phases. The following table summarizes the main files and responsibilities handled by each contributor.

Team Member	Main Contributions	Main Files
Adam TAWFIK	Parsing logic, hardware communication, client-side interaction, main application orchestration	`parser.py`, `meshtastic_comm_hw.py`, `client.py`, `main.py`
Arhan UNAY	Database layer, room management logic, simulation interface, logging, maintenance utilities	`database.py`, `room_manager.py`, `meshtastic_comm.py`, `logger.py`, `reset_db.py`

6.5 Lines of Code and Implementation Volume

To provide a measurable overview of the development effort, we estimated the size of the main Python modules produced during the project. The following table summarizes the implementation volume for each file.

File	Total Lines	Approx. Code Lines
`main.py`	32	13
`database.py`	159	135
`room_manager.py`	292	193

File	Total Lines	Approx. Code Lines
`parser.py`	38	26
`meshtastic_comm_hw.py`	162	109
`meshtastic_comm.py`	64	49
`client.py`	107	64
`reset_db.py`	59	38
`logger.py`	32	17
Total	**945**	**644**

Overall, the project represents approximately **945 total lines** and about **644 effective lines of code** across the main implementation files. This gives a concrete indication of the software development effort carried out during the project.

6.6 Persistence strategy

SQLite was selected as the storage engine instead of a flat JSON file. The main reason is reliability: SQLite supports atomic transactions and is more resilient when a write operation is interrupted by a crash or power cut. This directly supports the project requirement that rooms and messages must remain consistent after an abrupt shutdown.

The core schema includes a rooms table and a messages table. The rooms table stores room identity and metadata, while the messages table stores the room reference, sender node identifier, timestamp, and message content. This relational split is simple enough for embedded use and sufficient for the demonstrator scope.

7. Implementation Details

7.1 Command model

The command interface was intentionally kept simple. Commands start with the /room prefix and are followed by a subcommand and the required arguments. Examples of supported commands include /room create, /room list, /room post, and /room read. This choice reduces user-side complexity and keeps the parser lightweight.

7.2 Sequence of a message posting operation

Figure 5 - Detailed sequence diagram of a /room post command from the user interface to database insertion and response transmission.

The detailed sequence diagram shows the main data path for message posting. The packet first reaches the communication layer, then the parser verifies the syntax. The RoomManager checks the room state and triggers the database insertion. Once the transaction succeeds, a concise acknowledgement is built and returned to the requester through the LoRa communication chain.

7.3 LoRa-specific optimizations

- Replies are kept concise to fit within LoRa payload limitations and to reduce time-on-air.
- Room history reading is intentionally limited so that the network is not saturated by long responses.
- Longer content is segmented into shorter chunks; this reduces the probability of loss and improves readability on constrained devices.
- Administrative replies are sent as private messages when appropriate to avoid cluttering the public channel.

These optimizations are not cosmetic. They are directly linked to the radio constraints of LoRa: low throughput, limited payload size, and the risk of packet collision or loss when messages become too long.

7.4 Deployment and operational robustness

After local development, the software suite was deployed onto the Raspberry Pi. A systemd service was configured so that the application starts automatically at boot and restarts after failure. This approach improves availability and aligns with the requirement that the Room Server must remain operational even after abrupt events.

A dedicated private channel was also configured to isolate the demonstrator from unrelated public traffic. This reduced interference during experiments and improved the stability of the validation sessions.

8. Tests, Validation, and Observed Results

8.1 Validation strategy

Validation was carried out progressively, first in a local mock environment and then on real hardware. The objective was not only to verify nominal behavior but also to confirm that the system remained coherent after reboot, database reset, USB disconnection, and background execution.

8.2 Main test scenarios

Scenario	Goal	Expected result	Observed status
Create room	Check basic command execution	Server acknowledges room creation	Validated
Post message	Check storage path	Message stored in the selected room	Validated
Read history	Check retrieval and formatting	Recent messages returned in concise form	Validated
Restart persistence	Check data survival after reboot	Rooms and messages remain available	Validated
USB disconnection	Check recovery strategy	Server exits cleanly and restarts under systemd	Validated
Private channel configuration	Check isolation from external traffic	Only correctly configured nodes interact	Validated

8.3 Limits of the validation campaign

The validation campaign confirmed the functional objectives of the demonstrator, but it remained mostly qualitative rather than statistically exhaustive. For example, we did not produce a large dataset of latency measurements under many simultaneous clients. This

limitation is mainly due to time constraints and the practical complexity of repeating controlled radio experiments.

8.4 What was effectively demonstrated

Despite these limits, the project reached a meaningful maturity level. The team demonstrated that a Raspberry Pi-based node can extend Meshtastic with persistent rooms, command-based interaction, and controlled recovery after failure. This validates the core engineering idea behind the Room Server.

9. Impacts, Lessons Learned, and Future Work

9.1 Project impacts

From a technical point of view, the project demonstrates a practical use case for LoRa mesh networking beyond plain broadcast messaging. By adding a room-oriented application layer, the system makes the network more usable for structured exchanges. The meeting report also emphasized the potential relevance of such a service in isolated zones or emergency contexts where conventional telecommunications may be unavailable.

9.2 Lessons learned

- Embedded projects are strongly influenced by hardware details such as power supply quality and serial device behavior.
- Low-bandwidth systems require concise protocols and disciplined message design.
- A modest but reliable architecture is often preferable to a feature-rich but fragile implementation.
- Continuous reporting makes it easier to explain design choices and justify trade-offs at the end of the project.

9.3 Possible extensions

- User rights and administration roles for private or restricted rooms.
- Local web interface hosted on the Raspberry Pi.
- Multiple Room Server synchronization or replication.
- Usage statistics and monitoring dashboard.
- More quantitative performance measurements under controlled radio conditions.

10. Conclusion

This project achieved its main goal: designing and implementing a Meshtastic Room Server able to manage rooms, store message history, and keep its state after restart. The resulting demonstrator combines a Raspberry Pi, a Wio-E5 LoRa module, a modular Python code base, and an SQLite database into a coherent embedded communication service.

The work also illustrates an important engineering principle: in constrained environments, system value does not come from complexity but from choosing the right compromises. By focusing on persistence, robustness, concise communication, and deployment realism, the project produced a result that is both technically relevant and demonstrable.

Bibliography and Sources

- Project brief: Meshtastic LoRa Room Server subject sheet.
- Technical architecture note and implementation guide produced during the project.
- Weekly progress reports and client meeting report.
- Meshtastic tooling and command-line usage observed during deployment and testing.
- Project management writing guidelines provided by the supervisor.

The references above correspond to the internal project documents and tools that directly informed this report. Additional external technical documentation may be added later if a formal bibliography in APA style is required for the final submission package.

Appendices

Appendix A - Example command set

```
/room create ProjectAlpha
/room list
/room post ProjectAlpha Hello Team
/room read ProjectAlpha 5
```

Appendix B - Main deliverables

- Python source code for the Room Server and test client.
- Technical architecture note.
- Weekly progress reports.
- Mid-project client meeting report.
- Flyer, slides, and final report.